



Πανεπιστήμιο Πειραιώς

Σχολή Τεχνολογιών Πληροφορικής και Τηλεπικοινωνιών  
Τμήμα Ψηφιακών Συστημάτων  
Προπτυχιακό Πρόγραμμα Σπουδών

**Συστήματα Ευφύων Πρακτόρων**

Καθηγητής: Γεώργιος Βούρος

*Βοηθητικό Προσωπικό: Ναταλία Κολιού, Χρήστος Κυριαζόπουλος*

*2<sup>η</sup> Ομαδική Εργασία (2023-2024)*

Alexander Sviridov

[alex.sviridov123@gmail.com](mailto:alex.sviridov123@gmail.com)

E14166

Πειραιάς

26/2/2024



## Εισαγωγή

Στην παρούσα εργασία, προτείνεται ένα σύστημα, μαζί με την υλοποίηση αυτού στο πλαίσιο της Jason, δύο (2) πρακτόρων και του περιβάλλοντός τους.

Η παρούσα εργασία είναι η δεύτερη σε σειρά εργασία του μαθήματος «Συστήματα Ευφυών Πρακτόρων» και αποτελεί συνέχεια της πρώτης. Η πρώτη εργασία περιλαμβάνεται στον παραδοτέο φάκελο, ως πηγή αναφοράς για τον αναγνώστη.

Το μοντέλο (*model*) έχει παραμείνει όπως αυτό της πρώτης, με μερικές μικρές αλλαγές όπως την συμπερίληψη παρουσίας πολλαπλών ρομπότ στο περιβάλλον, αντί μονάχα του ενός.

Το περιβάλλον (*Env*) επίσης δεν διαφέρει σημαντικά από αυτό της πρώτης εργασίας. Η κύρια διαφορά είναι αυτή του ότι, πλέον το περιβάλλον ενημερώνει ξεχωριστά τον εκάστοτε πράκτορα κατόπιν δράσης του με τις αντίστοιχες αντιλήψεις. Κάθε φορά που λαμβάνει μια δράση ενός πράκτορα ενδέχεται να εμφανίσει νέο πελάτη και ενημερώνει έναν εκ των δύο πρακτόρων τυχαία για την εμφάνιση του νέου πελάτη, εάν αυτός πράγματι εμφανιστεί. Για αποφυγή φαινομένων deadlock, εντός της κλάσης *Env* έχει τεθεί μια ειδική μεταβλητή, *lock*, η οποία επιτρέπει την εμφάνιση μονάχα ενός νέου πελάτη ανά κύκλο αλληλοεπίδρασης. Επιπλέον, το περιβάλλον τίθεται να είναι κεντροποιημένο ή συγχρονισμένο, δηλαδή, αναμένει κάποια δράση και από τους δύο πράκτορες προτού προβεί στον επόμενο κύκλο αλληλοεπίδρασης με αυτούς.

Η αλλαγές που αξίζει να σημειωθούν και να αναλυθούν αφορούν κυρίως το ίδιο το μοντέλο της αρχιτεκτονικής του πράκτορα, το οποίο έχει αναθεωρηθεί για να λειτουργεί συνεργατικά με άλλον έναν όμοιο πράκτορα.

Συνεπώς, στην αναφορά αυτή θα περιγραφεί πρώτα το αναθεωρημένο μοντέλο της αρχιτεκτονικής του πράκτορα. Στη συνέχεια, θα επεξηγηθεί η προσομοίωση της εκτέλεσης του συστήματος και τέλος παρουσιάζονται τα πειραματικά αποτελέσματα και γίνεται σύγκριση αυτών με τα αντίστοιχα αποτελέσματα του συστήματος της πρώτης εργασίας.

## Περιεχόμενα

|                                                   |    |
|---------------------------------------------------|----|
| Γνωριμία.....                                     | 5  |
| Από Κοινού Χαρτογράφηση.....                      | 5  |
| Διαμοιρασμός Πελατών – Contract Net Protocol..... | 7  |
| Ανάθεση Λίστας Πελατών .....                      | 10 |
| Εκτέλεση της Προσομοίωσης .....                   | 12 |
| Πειραματικά Αποτελέσματα .....                    | 13 |
| Βιβλιογραφία.....                                 | 17 |

## Γνωριμία

Σε πρώτη φάση, οι πράκτορες πρέπει να γνωρίζουνε ο ένας το αναγνωριστικό όνομα του άλλου. Οπότε, ο πρώτος στόχος των πρακτόρων πλέον είναι αυτός της γνωριμίας, *!meet*.

Κατά αυτήν, ο κάθε πράκτορας εκπέμπει το όνομά του. Όταν ληφθεί η εκπομπή από κάποιον πράκτορα, το όνομα που περιλαμβάνεται στην εκπομπή αποθηκεύεται στη βάση γνώσης όποιου πράκτορα έλαβε την εκπομπή ως η πεποίθηση *agent(Ag)*.

*!meet.*

@meet

```
+!meet : .my_name(Me) <-  
        .broadcast(tell, agent(Me)).
```

## Από Κοινού Χαρτογράφηση

Για την ελαχιστοποίηση των συνολικών συγκρούσεων των ρομπότ με τοίχους, προστίθεται το στοιχείο της επικοινωνίας μεταξύ των πρακτόρων σχετικά με το ποιες μετακινήσεις είναι ασφαλείς και ποιες όχι.

Ο πράκτορας του συστήματος της πρώτης εργασίας, κατόπιν κάθε δράσης μετακίνησης, αποκτά τον στόχο του να ελέγχει το αποτέλεσμα της μετακίνησης. Εδώ, στο σχέδιο της πραγματοποίησης ελέγχου μετακίνησης, ο πράκτορας θα μεταβιβάζει το αποτέλεσμα του ελέγχου στον ισότιμό του. Έτσι, εάν δεν γίνει κάποιο σφάλμα επικοινωνίας ο άλλος πράκτορας δεν θα συγκρουστεί με τον ίδιο τοίχο, εκτός και αν η πρόσκρουση γίνει ταυτόχρονα και από τους δύο πράκτορες. Επίσης, εάν ένας πράκτορας έχει ανακαλύψει μια ασφαλή διαδρομή, θα την μαθαίνει μέσω της επικοινωνίας αυτής και ο άλλος πράκτορας.

Συγκεκριμένα, η επικοινωνία αυτή γίνεται με εκπομπή είτε της έκφρασης *blocked\_cell\_direction(Cell,Dir)* αν η μετακίνηση αποτέλεσε σε σύγκρουση, είτε *free\_cell\_direction(Cell,Dir)*, εάν η μετακίνηση ήταν επιτυχής.

```
free_cell_directions([]). blocked_cell_directions([]).  
num_of_free_moves(1). num_of_collisions(1).  
plan([]).
```

@check\_collision[atomic]

```
+!check : robot(Cell, premove) & robot(Cell) & direction(Dir) &  
        blocked_cell_directions(BCDs) &
```

```

        not .member([Cell, Dir], BCDs) &
        num_of_collisions(NoCs) <-
        .concat(BCDs, [[Cell, Dir]], NewBCDs);
        -+blocked_cell_directions(NewBCDs);
        -+num_of_collisions(NoCs + 1);
        .broadcast(tell, blocked_cell_direction(Cell, Dir));
        !plan.

```

```

@check_free[atomic]
+!check : robot(Cell, premove) & direction(Dir) &
        free_cell_directions(FCDs) &
        not .member([Cell, Dir], FCDs) &
        num_of_free_moves(NoFMs) <-
        .concat(FCDs, [[Cell, Dir]], NewFCDs);
        -+free_cell_directions(NewFCDs);
        -+num_of_free_moves(NoFMs + 1);
        .broadcast(tell, free_cell_direction(Cell, Dir)).
+!check.

```

Κατά την λήψη της εκπομπής οποιουδήποτε εκ των παραπάνω μηνυμάτων, ο πράκτορας παραλήπτης πραγματοποιεί την ίδια εσωτερική δράση με το εάν ο ίδιος είχε πραγματοποιήσει την ίδια μετακίνηση. Δηλαδή, ενημερώνει την δική του λίστα είτε μπλοκαρισμένων είτε ελεύθερων κατευθύνσεων κελιών και τον αντίστοιχο μετρητή. Στην περίπτωση εκπομπής περί μπλοκαρισμένης κατεύθυνσης, επανασχεδιάζει.

```

@blocked_cell_direction[atomic]
+blocked_cell_direction(Cell, Dir) : num_of_collisions(NoCs) &
        blocked_cell_directions(BCDs) &
        not .member([Cell, Dir], BCDs) <-
        .concat(BCDs, [[Cell, Dir]], NewBCDs);
        -+blocked_cell_directions(NewBCDs);
        -+num_of_collisions(NoCs + 1);
        !plan.

```

```

@free_cell_direction[atomic]
+free_cell_direction(Cell, Dir) : num_of_free_moves(NoFMs) &
        free_cell_directions(FCDs) &
        not .member([Cell, Dir], FCDs) <-
        .concat(FCDs, [[Cell, Dir]], NewFCDs);
        -+free_cell_directions(NewFCDs);
        -+num_of_free_moves(NoFMs + 1).

```

## Διαμοιρασμός Πελατών – Contract Net Protocol

Οι πράκτορες θα επιτυγχάνουν καλύτερο αποτέλεσμα εάν αντί απλώς να διατηρούν την τυχαία ανατεθείσα συλλογή διαδρομών πελατών, μπορούν να ανταλλάσσουν κατάλληλα τις διαδρομές πελατών που θα εξυπηρετήσουν.

Έστω οι δύο πράκτορες του συστήματος,  $ag_1$  και  $ag_2$ . Ο αλγόριθμος σχεδιασμού δράσης, ο οποίος είναι κοινός και για τους δύο πράκτορες, έχει τροποποιηθεί ώστε να επιστρέφει και το ρίσκο του πλάνου. Έστω οι τρέχουσες λίστες των προς εξυπηρέτηση διαδρομών του εκάστοτε πράκτορα,  $RT_1$  και  $RT_2$ . Το ερώτημα είναι σε ποιόν εκ των δύο πρακτόρων θα ήταν καλύτερο το να ανατεθεί μια νέα διαδρομή πελάτη  $R$ , υπό την τρέχουσα κατάσταση.

Απλοποιώντας τις εκφράσεις, έστω ότι το ρίσκο του πλάνου του κάθε πράκτορα, για τις τρέχουσες λίστες διαδρομών τους αντίστοιχα  $risk_1 := risk(RT_1)$  και  $risk_2 := risk(RT_2)$ .

Έστω τα ρίσκα των πλάνων για τις εναλλακτικές λίστες διαδρομών, δηλαδή αυτές με θα περιείχαν την νέα διαδρομή  $R$ ,  $altRisk_1 := risk(RT_1 \cup \{R\})$  και  $altRisk_2 := risk(RT_2 \cup \{R\})$ .

Τότε, η διαφορά του ρίσκου υπό την συμπερίληψη στον εκάστοτε πράκτορα θα είναι:

$$riskDiff_i := altRisk_i - risk_i, \quad i = 1, 2$$

Για παράδειγμα, έστω ότι για τον πράκτορα  $ag_1$  το ρίσκο για το τρέχον πλάνο του είναι  $risk_1 = -4$  και το ρίσκο του πλάνου όταν συμπεριληφθεί μια διαδρομή  $R$  προκύπτει να είναι ίσο με  $altRisk_1 = -6$ , τότε το διαφορικό ρίσκου είναι  $riskDiff_1 = -6 - (-4) = -2$ , δηλ., προκύπτει ότι το αναμενόμενο κέρδος είναι  $+2$  εάν προστεθεί η  $R$  στην λίστα του  $ag_1$ .

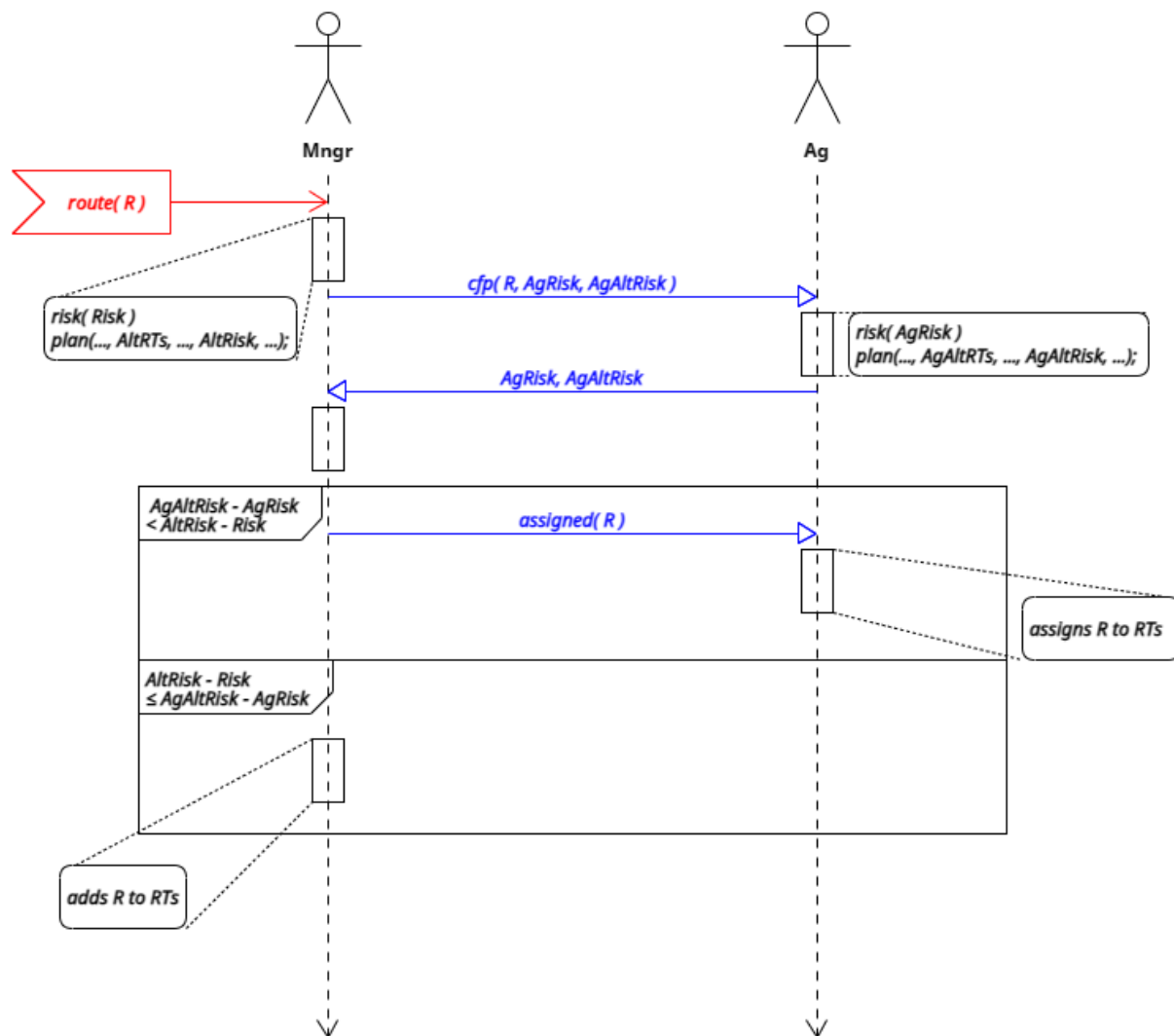
Έστω, επίσης, ότι για τον πράκτορα  $ag_2$  το ρίσκο για το τρέχον πλάνο του είναι  $risk_2 = -2$  και το ρίσκο του πλάνου όταν συμπεριληφθεί μια διαδρομή  $R$  προκύπτει να είναι ίσο με  $altRisk_2 = -5$ , τότε το διαφορικό ρίσκου είναι  $riskDiff_2 = -5 - (-2) = -3$ , δηλ., προκύπτει ότι το αναμενόμενο κέρδος είναι  $+3$  εάν προστεθεί η  $R$  στην λίστα του  $ag_2$ .

Στην περίπτωση του παραδείγματος, είναι καλύτερο, υπό τις τρέχουσες πεποιθήσεις των πρακτόρων το να αποδοθεί τελικώς η διαδρομή στον  $ag_2$ , αφού με αυτήν το κέρδος είναι 3.

Στο σύστημα της εργασίας αυτής, αντί ο πράκτορας να εισάγει τη νέα διαδρομή άμεσα στην λίστα των προς εξυπηρέτηση διαδρομών του  $RTs$ , της πεποίθησης  $routes(RTs)$ , θα πρέπει πρώτα να κάνει τον παραπάνω συλλογισμό. Ο πράκτορας που πραγματοποιεί αυτή την διαδικασία αναφέρεται ως ο διαχειριστής (*Mngr*) της νέας διαδρομής  $R$ . Ο *Mngr* θα πρέπει να έχει διαθέσιμο το δικό του διαφορικό ρίσκου και αυτό του άλλου πράκτορα,  $Ag$ . Για να

αποκτήσει το διαφορικό ρίσκου του  $Ag$  θα πρέπει να επικοινωνήσει με αυτόν, ζητώντας του είτε το ίδιο το διαφορικό είτε το τρέχον και το εναλλακτικό του ρίσκο για να το συνάγει.

Ως προς το Contract Net Protocol που εφαρμόζεται στην παρούσα εργασία, παρατίθεται το ακόλουθο διάγραμμα ακολουθίας.



Διάγραμμα Ακολουθίας του Contract Net Protocol του Συστήματος

Η διαδικασία ξεκινά όταν ένας πράκτορας λάβει μια νέα αντίληψη  $route(R)$ , όπου ο πράκτορας πλέον θα δράσει ως διαχειριστής της διαδρομής  $R$ . Αρχικά, ο διαχειριστής υπολογίζει εναλλακτικό του ρίσκο,  $AltRisk$ , και ανακτά το τρέχον ρίσκο του,  $Risk$ , μέσω της πεποιθήσης  $risk(Risk)$ , η οποία αρχικά είναι μηδέν και υπολογίζεται κάθε φορά που ο πράκτορας σχεδιάζει το πλάνο του.

Στη συνέχεια, αποστέλλει ένα μήνυμα cfp (call for proposal) στον πράκτορα *Ag*, όπου ως απόκριση θα αναμένει τις τιμές του τρέχοντος και εναλλακτικού ρίσκου του *Ag*, *AgRisk* και *AgAltRisk*, αντίστοιχα.

```
@route
+route(R) : agent(Ag) & risk(Risk) &
            free_cell_directions(FCDs) &
            blocked_cell_directions(BCDs) &
            nodes(NDs) & routes(RTs) & carry(C) & energy(E) &
            num_of_free_moves(NoFMs) & num_of_collisions(NoCs) <-

            -+managed_route(R);

            .concat(RTs, [R], AltRTs);
            agt.planner.plan(
                FCDs, BCDs, NDs, AltRTs, C, E, NoFMs, NoCs, AltRisk, AltPL);

            .send(Ag, askOne,
                cfp(R, AgAltRisk, AgRisk), cfp(R, AgAltRisk, AgRisk));
            !assing(Ag, AgAltRisk, AgRisk, AltRisk, Risk).
```

Κατά την λήψη του προς απάντηση μηνύματος cfp, ο πράκτορας *Ag* αποκτά τον στόχο ερωτήματος *+?cfp* και πραγματοποιεί σχεδιασμό πλάνου για την εναλλακτική λίστα διαδρομών, που να περιλαμβάνει την *R*, και αποκρίνεται με το ρίσκο αυτού του εναλλακτικού πλάνου καθώς και του τρέχοντος ρίσκου του.

```
@cfp
+?cfp(R, AltRisk, Risk) :
    risk(Risk) &
    free_cell_directions(FCDs) & blocked_cell_directions(BCDs) &
    nodes(NDs) & routes(RTs) & carry(C) & energy(E) &
    num_of_free_moves(NoFMs) & num_of_collisions(NoCs) <-

    .concat(RTs, [R], AltRTs);
    agt.planner.plan(
        FCDs, BCDs, NDs, AltRTs, C, E, NoFMs, NoCs, AltRisk, AltPL).
```

Όταν ο διαχειριστής της *R* λάβει την απάντησή του, αποκτά τον στόχο του να αναθέσει την διαδρομή *R* και προβαίνει στο να συγκρίνει τις διαφορές και να αποφανθεί στο ποιόν θα αναθέσει την *R*, ως εξής:

- Αν  $AltRisk - Risk \leq AgAltRisk - AgRisk$ , τότε αναθέτει την *R* στον εαυτό του.
- Ειδάλλως αναθέτει την *R* στον *Ag*.

```

@assing_ag
+!assing(Ag, AgAltRisk, AgRisk, AltRisk, Risk) :
    AgAltRisk - AgRisk < AltRisk - Risk & managed_route(R) <-

    .send(Ag, tell, assigned(R)).

@assing_me[atomic]
+!assing(Ag, AgAltRisk, AgRisk, AltRisk, Risk) : managed_route(R) &
    routes(RTs) <-

    .concat(RTs, [R], NewRTs);
    -+routes(NewRTs);
    !plan.

```

Και τέλος, αν η ανάθεση γίνει στον πράκτορα *Ag*, αυτός προσθέτει την *R* στην λίστα των διαδρομών και επανασχεδιάζει.

```

@assigned[atomic]
+assigned(R) : routes(RTs) <-
    .concat(RTs, [R], NewRTs);
    -+routes(NewRTs);
    .print("+assigned(", R, "), NewRTs: ", NewRTs);
    !plan.

```

## Ανάθεση Λίστας Πελατών

Σε ορισμένα σενάρια, ένας πράκτορας ενδέχεται να μην έχει την απαιτούμενη ενέργεια για να εξυπηρετήσει επιπλέον διαδρομές πελατών της λίστας διαδρομών του, ενώ ταυτόχρονα ο άλλος πράκτορας να έχει την δυνατότητα να εξυπηρετήσει και πελάτες της λίστας του εξασθενημένου πράκτορα. Σε τέτοιες περιπτώσεις ίσως είναι συνετό το να ανατεθούν οι διαδρομές του πράκτορα που αδυνατεί να πραγματοποιήσει περεταίρω εξυπηρετήσεις στον πράκτορα που μπορεί.

Ο αλγόριθμος αναζήτησης όπως είχε στην προηγούμενη εργασία υπολόγιζε την βέλτιστη ακολουθία εξυπηρετήσεων. Εάν δεν εύρισκε κάποιο πλάνο με αρνητικό ρίσκο, δηλαδή θετικό αναμενόμενο κέρδος, τότε επέστρεφε πλάνο που περιείχε μόνο τον κωδικό δράσης αναμονής, με μηδενικό ρίσκο. Αυτό δεν σημαίνει πως ο πράκτορας απαραίτητα θα αδυνατεί να δράσει, το πλάνο μηδενικού ρίσκου απλώς σήμαινε ότι από άποψη ρίσκου ήταν καλύτερα να μην δράσει ο πράκτορας.

Θα ήταν όμως επιθυμητό, ο αλγόριθμος να επέστρεφε μηδενικό ρίσκο μονάχα αν πράγματι ο πράκτορας δεν μπορεί να δράσει. Ειδικά να επιστρέφει θετικό ρίσκο αν μπορεί να δράσει αλλά δεν συμφέρει, και αρνητικό ρίσκο αν η δράση έχει θετικό αναμενόμενο κέρδος.

Ο αλγόριθμος αναζήτησης πράγματι θα επιστρέφει μαζί και το αντίστοιχο πλάνο, αλλά προτού επιστραφεί το πλάνο στον πράκτορα εάν το ρίσκο είναι θετικό το πλάνο αντικαθίσταται από τον κωδικό της δράσης αναμονής (9), από τον σχεδιαστή του πλάνου.

Οπότε, πλέον, ο σχεδιαστής πλάνου αλλάζει ώστε να συμπεριφέρεται σύμφωνα με την ακόλουθη έκφραση:

$$planner.plan \rightarrow \begin{cases} < 0, & \text{αν έχει βρεθεί πλάνο με θετικό αναμενόμενο κέρδος} \\ = 0, & \text{αν δεν υπάρχει δυνατότητα εξυπηρέτησης} \\ > 0, & \text{αλλιώς} \end{cases}$$

Στην τελευταία περίπτωση εννοείται πως αν και ο πράκτορας δεν αδυνατεί να εξυπηρετήσει πελάτες δεν έχει βρεθεί κάποιο πλάνο με θετικό κέρδος όπου το πλάνο θα έχει ξανά μονάχα τη δράση αναμονής.

Ο σχεδιαστής επιστρέφει μηδενικό ρίσκο αν δεν υπάρχει δυνατότητα εξυπηρέτησης, το οποίο γίνεται υπό δύο περιπτώσεις. Η πρώτη είναι ο πράκτορας να μην έχει καμία διαδρομή στην λίστα των διαδρομών του, ενώ η δεύτερη πως το ρομπότ που διαχειρίζεται ο πράκτορας δεν έχει επαρκή ενέργεια για οποιαδήποτε εξυπηρέτηση.

Άρα αυτή είναι η συνθήκη για την ανάθεση της λίστας των διαδρομών εξασθενημένου πράκτορα στον άλλον. Δηλαδή, για  $risk(Risk)$ ,  $Risk == 0$  και  $not .length(Routes, 0)$ . Κάθε φορά που ο πράκτορας αποκτά τον στόχο της αναμονής εξετάζει εάν η παραπάνω είναι αληθής και αν ναι αναθέτει όλες τις διαδρομές του στον άλλον πράκτορα, και τις διαγράφει.

```
+!wait : (routes(RTs) & .length(RTs, 0)) <-  
    wait; !wait.
```

```
@wait_trans[atomic]  
+!wait : risk(Risk) & Risk == 0 & agent(Ag) & routes(RTs) <-  
    .send(Ag, tell, transfer(RTs));  
    -+routes([]);  
    !wait.
```

```
+!wait <- !plan; !work; !wait.
```

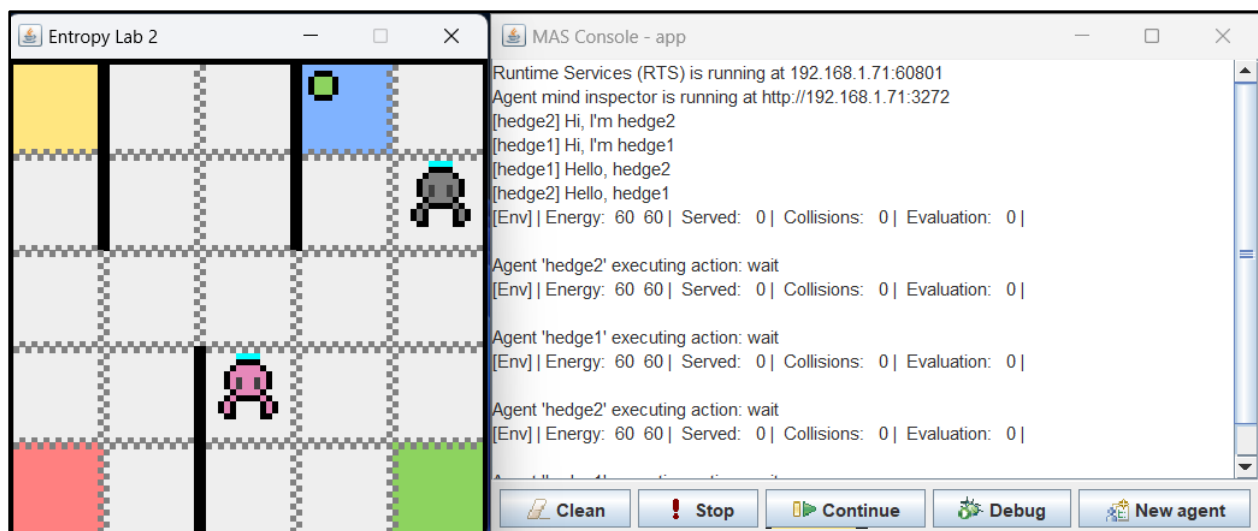
```
@transfer // [atomic]  
+transfer(TransRTs) : routes(RTs) <-  
    .concat(RTs, TransRTs, NewRTs);
```

```
-+routes(NewRTs);  
!plan.
```

## Εκτέλεση της Προσομοίωσης

Για να τρέξετε την εφαρμογή της προσομοίωσης, ακολουθείστε τα ίδια βήματα με αυτά της προηγούμενης εργασίας της ενότητας **Εκτέλεση της Προσομοίωσης**.

Εκκινώντας την εφαρμογή, ξανά εμφανίζεται το παράθυρο “Entropy Lab 2”, και το παράθυρο “MAS Console app”.



Το παράθυρο “MAS Console app” εδώ εκτυπώνει διαφορετικές πληροφορίες από αυτήν της πρώτης εργασίας, με την εξαίρεση του ρίσκου νέας μετακίνησης που έχει παραμείνει για να φαίνεται πως το σύστημα λειτουργεί ορθά. Επίσης παραμένει η εκτύπωση της τρέχουσας δράσης του εκάστοτε πράκτορα καθώς και της εκτίμησης στην οποία αναγράφονται οι διαθέσιμες ενέργειες των πρακτόρων, το πλήθος των εξυπηρετημένων πελατών, το πλήθος των συγκρούσεων και το τρέχον evaluation. Όπως φαίνεται παρακάτω

New-Move Risk: 9.090909

```
Agent 'hedge1' executing action: wait  
[Env] | Energy: 60 60 | Served: 0 | Collisions: 0 | Evaluation: 0 |
```

Αποκρύπτονται οι εμφανίσεις των πινάκων γειτνίασης, η ακολουθία εξυπηρέτησης και το πλάνο το κάθε πράκτορα, για εποπτικούς λόγους απλότητας. Κυρίως εκτυπώνονται οι ανταλλαγές μηνυμάτων μεταξύ των πρακτόρων. Το μήνυμα κατά την ανάθεση μιας διαδρομής από τον διαχειριστή της, εκτυπώνει τα διαφορικά και την αντίστοιχη ανάθεση.

```
[hedge2] MyDiff (0) VS AgDiff (-6)
[hedge2] ~~~> ASSIGNED [1,2] TO hedge1
[hedge1] +assigned([1,2]), NewRTs: [[3,1],[1,2]]
```

Ξανά, γεγονότα που συνυπολογίζονται στο evaluation εκτυπώνονται όπως φαίνεται στο ακόλουθο παράδειγμα:

```
Agent 'hedge1' executing action: move(2)
ACTION (-1), Evaluation: -6
COLLISION (-100), Evaluation: -106
[Env] | Energy: 54 60 | Served: 0 | Collisions: 1 | Evaluation: -106 |
```

Σε περίπτωση ανάθεσης όλης τις λίστας διαδρομών από έναν εξασθενημένο πράκτορα στον ενεργό πράκτορα, εκτυπώνεται αντίστοιχη αναπαράσταση όπως στο ακόλουθο παράδειγμα:

```
[hedge1] >>>>> Transferred routes.
[hedge2] <<<<< Received routes.
[hedge2] NewRTs: [[3,1],[4,3],[2,1],[1,2],[3,4],[3,2],[1,2],[1,1]]
```

Παρατήρηση: Στην εφαρμογή της προσομοίωσης του επεισοδίου, προκύπτουν με μια εμφανή συχνότητα εξαιρέσεις (exceptions), κυρίως σφάλματα επικοινωνίας (communication errors), ενώ σπάνια υπάρχουνε πιο σοβαρές εξαιρέσεις. Οι εξαιρέσεις κατά τις cfr επικοινωνίες προκύπτουν περίπου μια στις τέσσερις φορές. Αν και έγινε προσπάθεια κατά την ανάπτυξη του συστήματος για υλοποίηση ενός μηχανισμού επιβεβαίωσης, απλώς πρόσθετε επιπλέον πολυπλοκότητα και προκύπταν επιπλέον σφάλματα επικοινωνίας και deadlocks. Οπότε, δυστυχώς, τα σφάλματα επικοινωνίας έγιναν αποδεκτά και κατά κάποιον τρόπο μπορεί να ειπωθεί πως έτσι προσομοιώνεται καλύτερα ένα πραγματικό περιβάλλον.

## Πειραματικά Αποτελέσματα

Κατά τα πειράματα που διεξαχθήκαν, έγινε προσπάθεια να μην συμπεριληφθούν οι προσομοιώσεις που εμφάνιζαν σημαντικό βαθμό εξαιρέσεων ή άλλων σφαλμάτων. Πέρα τούτου, δεν έγινε καμία άλλη εξαγωγή λεγόμενων outliers. Οι προσομοιώσεις του συστήματος αυτής αλλά και της προηγούμενης εργασίας έγιναν 33 φορές για το κάθε μέγιστο πλήθος δράσεων ή μέγιστης ενέργειας (energy) που αποδόθηκε στους πράκτορες. Τα αποτελέσματα των προσομοιώσεων έχουν συμπεριληφθεί εντός του φακέλου `sim/`, το οποίο συμπεριλαμβάνεται εντός του παραδοτέου φακέλου. Στον φάκελο αυτό περιλαμβάνονται και οι αντίστοιχες προσομοιώσεις του συστήματος της πρώτης εργασίας. Οι μέσοι όροι αυτών των προσομοιώσεων παρατίθενται στον ακόλουθο πίνακα:

| <b>Max Robot Energies</b>  | <b>30 30</b> | <b>60 60</b> | <b>120 120</b> | <b>240 240</b> |
|----------------------------|--------------|--------------|----------------|----------------|
| <b>Avg. Energy Used</b>    | 54.12        | 115.15       | 234.79         | 463.45         |
| <b>Avg. Clients Served</b> | 6.27         | 13.58        | 26.39          | 47.18          |
| <b>Avg. Collisions</b>     | 3.61         | 4.03         | 4.09           | 3.97           |
| <b>Avg. Evaluation</b>     | -288.24      | -246.85      | -115.85        | +83.49         |

Στον παραπάνω πίνακα οι ενέργειες που αποδίδονται αναγράφονται ξεχωριστά επειδή ο κάθε πράκτορας διαχειρίζεται ένα εκ των δύο ισοδύναμα ρομπότ του περιβάλλοντος που έχουν ίσα ποσά ενέργειας (συνολικών διαθέσιμων δράσεων). Οπότε όταν αναγράφεται π.χ., 30 30 ως την επικεφαλίδα της 1<sup>ης</sup> στήλης δεδομένων, εννοείται ότι το συνολικό πλήθος διαθέσιμων δράσεων για το σύστημα είναι 60, αλλά το κάθε ρομπότ μπορεί να εκτελέσει το πολύ 30 δράσεις.

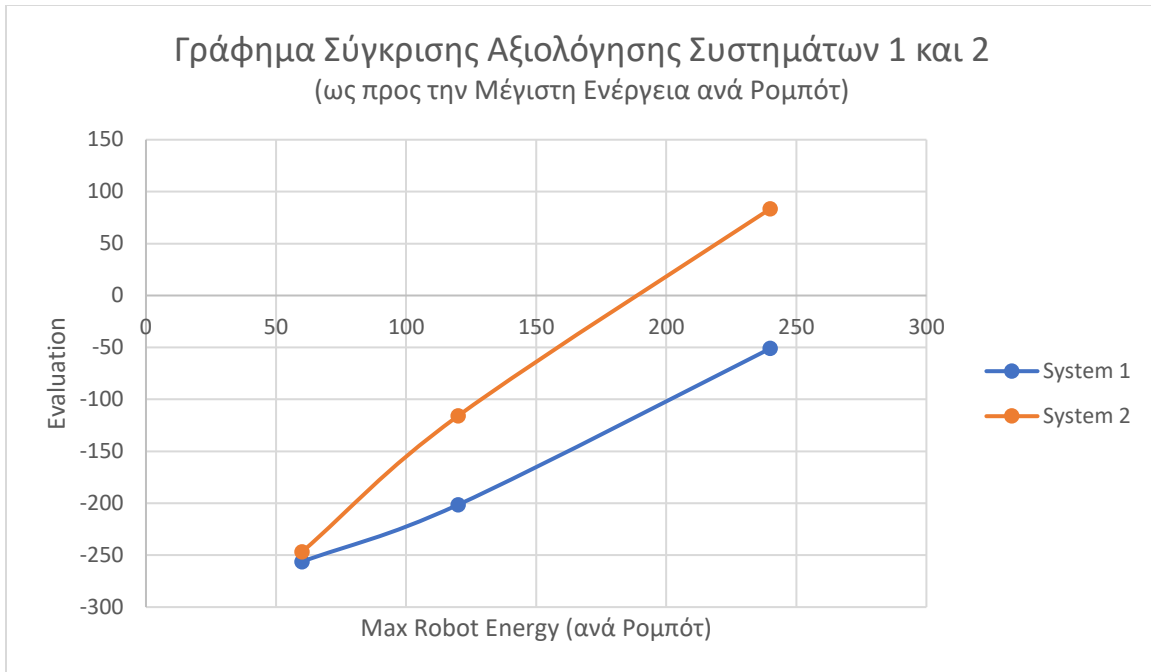
Για λόγους σύγκρισης παραθέτουμε ακολούθως και τον αντίστοιχο πίνακα μέσω όρων του μονοπρακτορικού συστήματος από την πρώτη εργασία (Σύστημα 1):

*Πίνακας Μέσω Όρων Προσομοιώσεων του Μονοπρακτορικού Συστήματος*

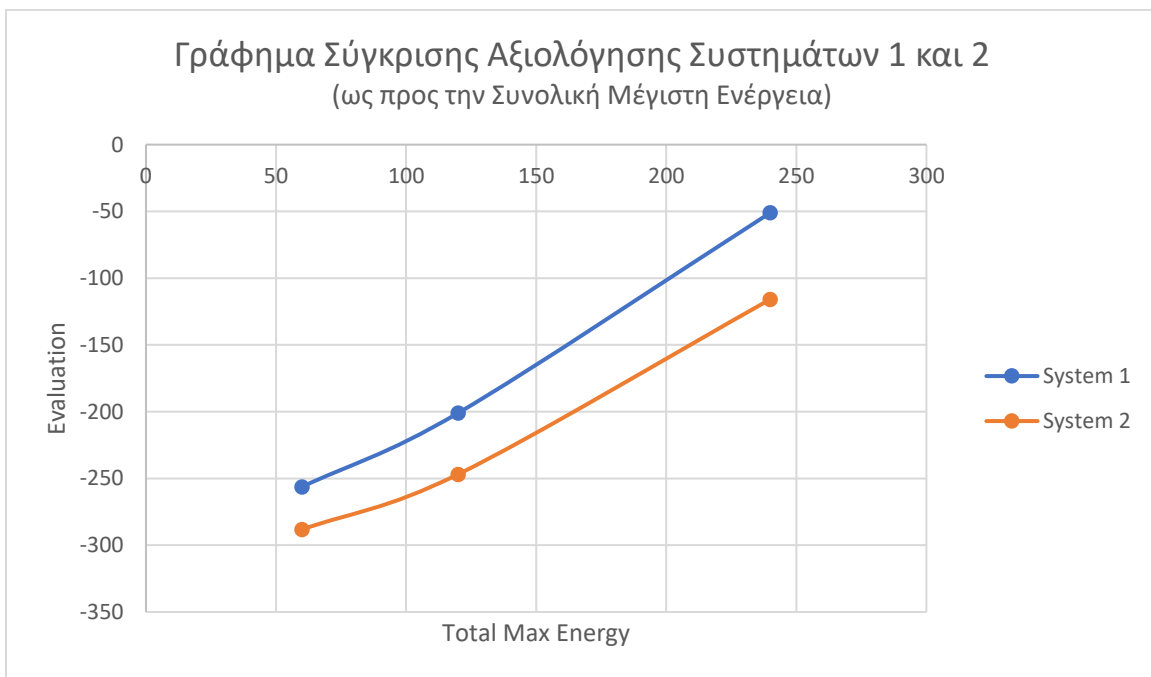
| <b>Max Robot Energy</b>    | <b>60</b> | <b>120</b> | <b>240</b> |
|----------------------------|-----------|------------|------------|
| <b>Avg. Energy Used</b>    | 58        | 117.6      | 237        |
| <b>Avg. Clients Served</b> | 8.42      | 16.12      | 29.9       |
| <b>Avg. Collisions</b>     | 3.7       | 4          | 4.12       |
| <b>Avg. Evaluation</b>     | -256.27   | -201.42    | -51        |

Υπάρχουνε δύο τρόποι σύγκρισης μεταξύ των δύο συστημάτων βάσει των παραπάνω αποτελεσμάτων. Ο πρώτος είναι να συγκρίνουμε τα δύο συστήματα θεωρώντας ισοδύναμα ρομπότ, δηλαδή εάν στο Σύστημα 1 είχαμε ένα ρομπότ με ενέργεια  $E$ , να το συγκρίνουμε με το Σύστημα 2 όπου και τα δύο ρομπότ έχουν το καθένα επίσης ενέργεια  $E$ . Τότε, είναι προφανές ότι το Σύστημα 2 υπερτερεί κατά πολύ, που είναι και το αναμενόμενο, μιας και το Σύστημα 2 διαθέτει πολλαπλάσιο εργατικό δυναμικό.

Η σύγκριση με αυτόν τον τρόπο παρουσιάζεται με την ακόλουθη γραφική παράσταση.



Όμως, μπορεί να υποστηριχθεί για τον λόγο του ότι το Σύστημα 2 έχει διπλάσιο εργατικό δυναμικό, πως αυτή η σύγκριση είναι άδικη. Εναλλακτικά, τα δύο συστήματα μπορούν να συγκριθούν βάσει της συνολικής ενέργειας που διαθέτουν όλα τα ρομπότ μαζί. Τότε, το αντίστοιχο διάγραμμα σύγκρισης είναι το παρακάτω:



Παρατηρείται πως το σύστημα δύο, υπό αυτό το κριτήριο και κατά τη δοθείσα αξιολόγηση, αποδίδει εμφανώς χειρότερα. Η ερμηνεία που προτείνεται εδώ για το γεγονός αυτό είναι

πως το Σύστημα, όπως έχει υλοποιηθεί στην παρούσα εργασία, αποδίδει καλύτερα ως μονοπρακτορικό, για συνολικά ίσα ποσά διαθέσιμων μονάδων ενέργειας. Προτείνονται τα ακόλουθα τρία αίτια που εξηγούν την τελευταία εικασία σε αύξουσα σειρά σημαντικότητας.

1. Αν και αποτελεί θέμα υλοποίησης, κατά την προσομοίωση προκύπτουν εξαιρέσεις επικοινωνίας, όπως προαναφέρθηκε στην προηγούμενη ενότητα. Αυτά τα σφάλματα οδηγούν σε χαμένες αναθέσεις πελατών, με αποτέλεσμα οι πράκτορες να μην έχουν επίγνωση για ορισμένους πελάτες. Το φαινόμενο των χαμένων πελατών είναι αρκετά μεγάλο πρόβλημα, μιας και κάθε πελάτης αποτελεί «ευκαιρία» για τον πράκτορα για να δημιουργήσει πλάνο με μεγαλύτερη «αλυσίδα» εξυπηρέτησης πελατών με μεγαλύτερο κέρδος. Ωστόσο, ακόμη και αν δεν υπήρχαν αυτές οι εξαιρέσεις επικοινωνίας, τα ακόλουθα δύο αίτια υφίστανται άνευ υλοποίησης, υπό την θεώρηση του προβλήματος που έχει διατυπωθεί και είναι ακόμη σημαντικότερα.
2. Το πρωτόκολλο CNP δεσμεύει τον πράκτορα στον οποίο γίνεται η ανάθεση στο να επιτελέσει την εξυπηρέτηση το εν λόγω πελάτη. Ωστόσο, η δέσμευση αυτή μπορεί να είναι πρόωρη, και ίσως αν ήταν καλύτερο το να ανατεθεί ο πελάτης σε άλλον πράκτορα. Αυτό βέβαια, θα απαιτούσε διαφορετική προσέγγιση. Οι πράκτορες θα έπρεπε να επανεξετάζουν και να συγκρίνουν τις λίστες των πελατών τους τακτικά ανταλλάσσοντας τους πελάτες βάσει του πώς εξελίσσεται το περιβάλλον. Μια τέτοια προσέγγιση θα απαιτούσε αρκετά μεγαλύτερο βαθμό πολυπλοκότητας, και ίσως ξεφεύγει από το πλαίσιο της εργασίας αυτής. Από την άλλη, στο μονοπρακτορικό σύστημα, ο πράκτορας διαχειρίζεται όλη τη λίστα πελατών, δεν υπάρχουν πελάτες που δεν μπορεί να εξυπηρετήσει επειδή έχουν ανατεθεί σε κάποια άλλη οντότητα. Αυτό είναι σημαντικό γιατί η αρχή που διέπει την καλή λειτουργία του συστήματος είναι στο να μην γίνεται «σπατάλη» της ενέργειας του ρομπότ, οπότε να ελαχιστοποιείται το πλήθος των διαδρομών από το ένα χρωματιστό κελί στο άλλο χωρίς ο πράκτορας να μεταφέρει ταυτόχρονα κάποιον πελάτη. Για να μπορεί ο πράκτορας να σχεδιάζει και να υλοποιεί τέτοια συνεχή δρομολόγια, θα πρέπει να διαθέτει είτε όλο το πελατολόγιο είτε ένα υποσύνολό του που να του δίνει την δυνατότητα να κατασκευάζει τέτοιες αλυσίδες από διαδρομές πελατών. Εάν λοιπόν η ανάθεση γίνεται με δεσμευτικό τρόπο κατά την εμφάνιση των πελατών, η δεσμεύσεις αυτές συχνά μπορούν να χαρακτηριστούν μη βέλτιστες μακροπρόθεσμα.
3. Όσοι περισσότεροι πράκτορες εξυπηρετούν ταυτόχρονα κατά το στάδιο «εξερεύνησης», δηλαδή το στάδιο στο οποίο οι πράκτορες ακόμη δεν έχουν καθορίσει ασφαλείς διαδρομές μεταξύ των χρωματιστών κελιών, τόσο μεγαλύτερο το ρίσκο σύγκρουσης. Αυτό έχει φανεί πειραματικά αλλά προκύπτει και διαισθητικά. Και βάσει το μεγάλου κόστους που έχει αποδοθεί στις συγκρούσεις, το μονοπρακτορικό σύστημα είναι περισσότερο ασφαλές.

Τα παραπάνω καλύπτουν την σύγκριση μεταξύ του συστήματος της 1<sup>ης</sup> και αυτής της εργασίας.

Να σημειωθεί ότι κατά το πειραματικό στάδιο παρατηρήθηκαν δύο ενδιαφέρουσες αναδυόμενες συμπεριφορές για το Σύστημα 2, οι οποίες δεν ήταν εσκεμμένες.

- Γενικότερα, οι πράκτορες τείνουν να «προτιμάνε» να εξυπηρετούν τους πελάτες που έχουν γνωστό ασφαλές δρομολόγιο πρώτα, καμιά φορά μέχρι να μην απομένει και κανένας με ασφαλή διαδρομή, προτού επιχειρήσουν τις μη ασφαλείς.
- Περισσότερο ενδιαφέρον παρουσίασαν τα πειράματα όπου οι πράκτορες είχαν μεγάλο ποσό ενέργειας (240 240), όπου οι πράκτορες έτειναν να αποδίδουν σε έναν μονάχα εκ των δυο τους όλους του πελάτες. Αυτό, αν και αντιβαίνει τη διαίσθηση, βάσει της παραπάνω ανάλυσης, είναι επωφελές για το σύστημα, μιας και καταλήγει έτσι το σύστημα να απολαμβάνει χαμηλότερο ρίσκο κατά το στάδιο της εξερεύνησης.

## Βιβλιογραφία

[1] Artificial Intelligence a Modern Approach Third Edition, Stuard Rusel and Peter Norvig

[2] An Introduction to MultiAgent Systems, Michael Wooldridge

[3] Programming multi-agent systems in AgentSpeak using Jason, Bordini, Hubner, Wooldirdge